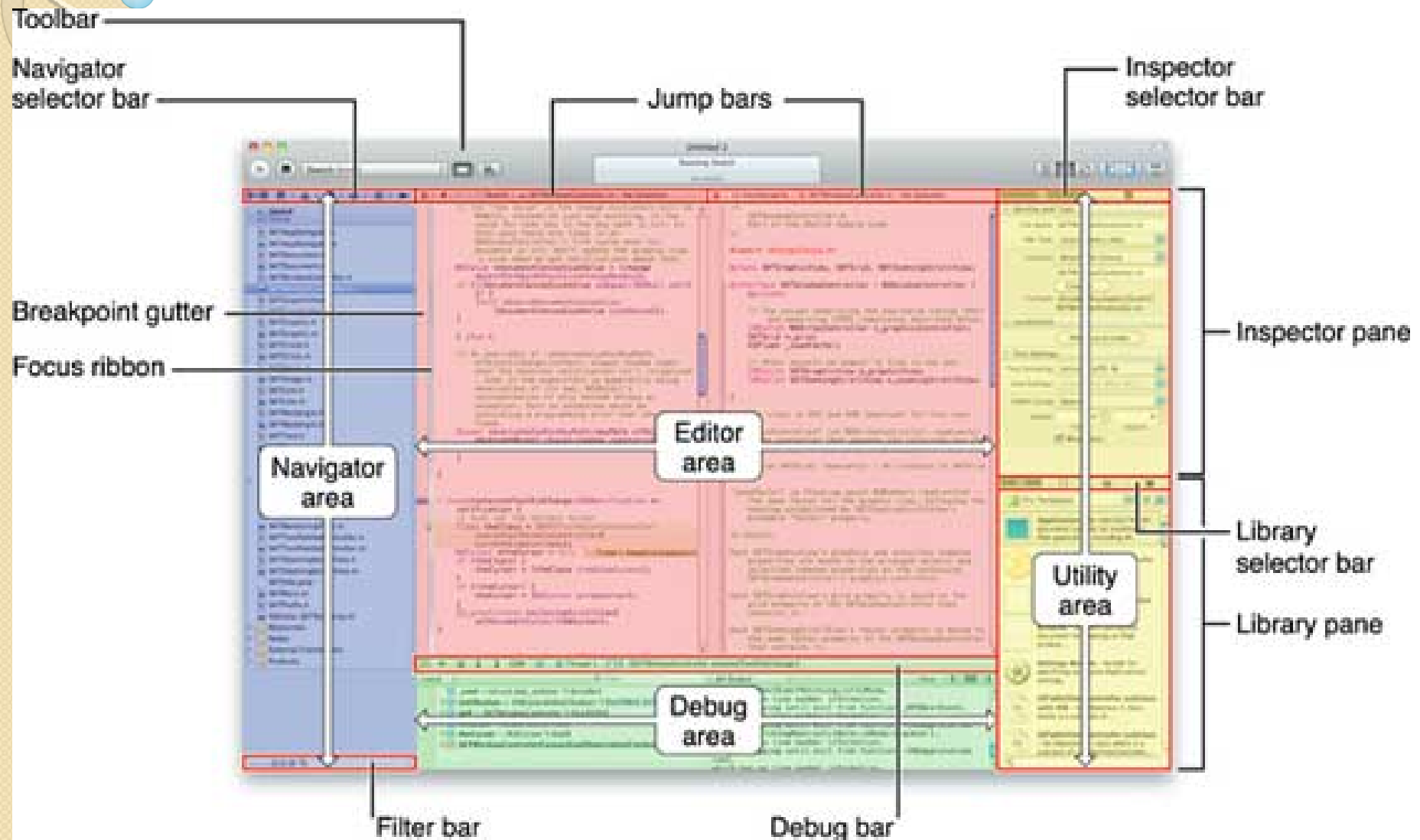


Xcode IDE



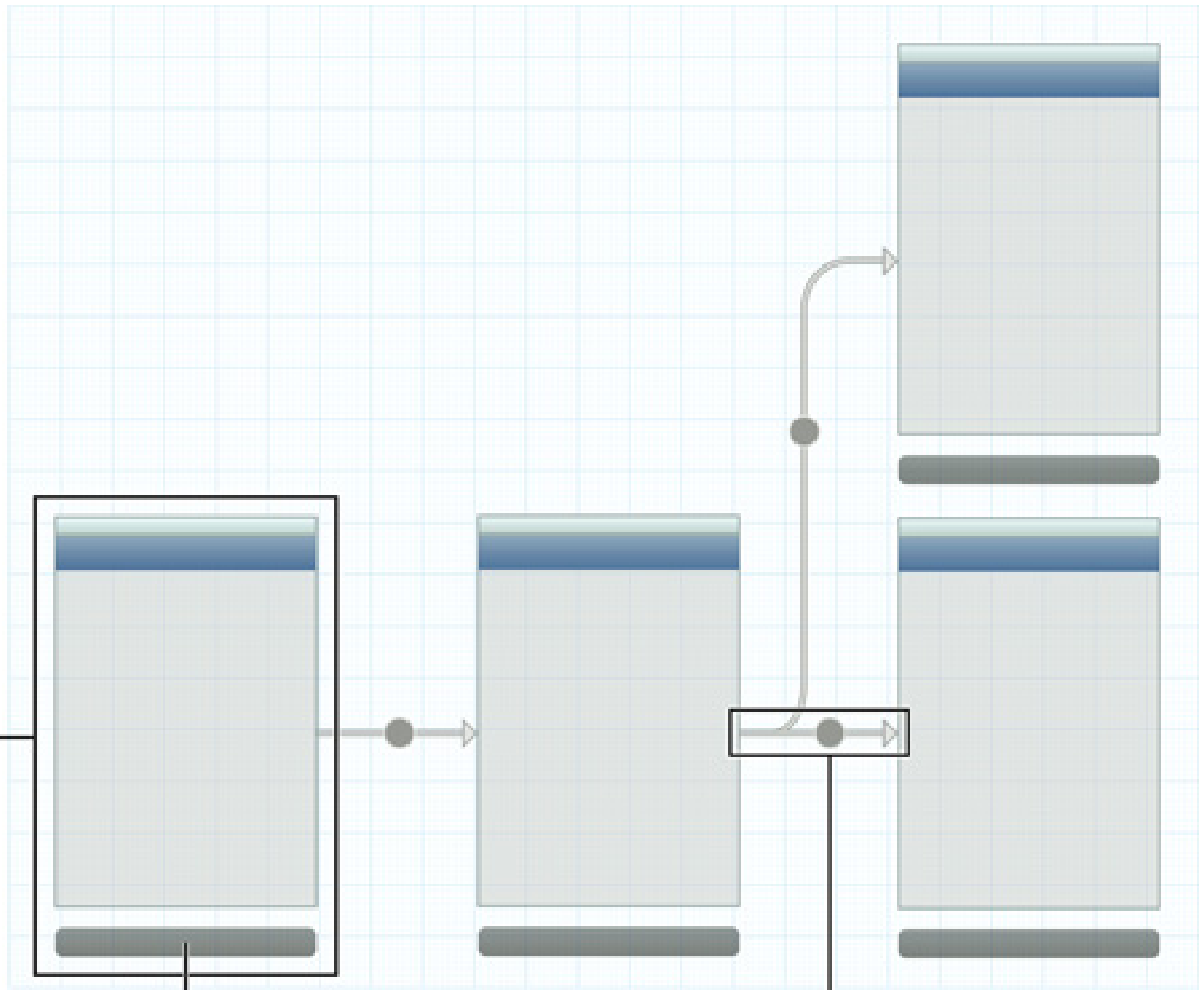
What is Storyboard?

- A storyboard is a visual representation of the user interface of an iOS application, showing screens of content and the connections between those screens.
- A storyboard is composed of a sequence of scenes, each of which represents a view controller and its views; scenes are connected by segue objects, which represent a transition between two view controllers.
- Storyboard help us to create all the screens of the application along with their flow of screens under one interface `MainStoryboard.storyboard`.

Scene

Dock

Segue



Advantages of Storyboard

- You have better conceptual overview of all the screens and connections between them.
- It describes the transition between the various screens.
- Make working with table and views a lot easier with the new prototype and static cells features.
- Can also use .xibs if needed.

Dis-advantages of Storyboard

- Need a big monitor.
- Only available in iOS 5 onward
- Merges can be very difficult.

What is Interface Builder?

- Interface Builder is the tool that enables easy creation of UI interface.
- The Interface Builder within Xcode makes it simple to design a full user interface without writing any code.
- Simply drag and drop windows, buttons, text fields, and other objects onto the design canvas to create a functioning iPhone, or iPad user interface.
- Because Cocoa and Cocoa Touch are built using the Model-View-Controller pattern, it is easy to independently design your interfaces, separate from their implementations.
- User interfaces are actually archived Cocoa or Cocoa Touch objects (saved as .nib files), and OS X and iOS will dynamically create the connection between UI and code when the app is run.

What is Outlet?

- An outlet is a property of an object that references another object. The reference is archived through Interface Builder.
- The connections between the containing object and its outlets are reestablished every time the containing object is unarchived from its nib file.
- The containing object holds an outlet declared as a property with the type qualifier of IBOutlet.

Outlet cont....

- An outlet is more than a simple property.
- The connection between an object and its outlets is archived in a nib file; when the nib file is loaded, each connection is unarchived and reestablished, and is thus always available whenever it becomes necessary to send messages to the other object.
- The type qualifier IBOutlet is a tag applied to an property declaration so that the Interface Builder application can recognize the property as an outlet and synchronize the display and connection of it with Xcode.



Views and Windows

Views and Windows

- windows and views to present your application's content on the screen
- Windows do not have any visible content themselves but provide a basic container for application's views.
- Every application has at least one window and one view for presenting its content.

UIView

- UIKit provides predefined views that you can use to present your content.
- These views range from simple buttons and text labels to more complex views such as table views, picker views, and scroll views.

Views Manage Application's Visual Content

- A view is an instance of the UIView class and manages a rectangular area in your application window.
- Views are responsible for :
 - drawing content
 - handling multitouch events
 - managing the layout of any subviews.

Responsibilities for Views

Layout and subview management

- A view defines its own default resizing behaviors in relation to its parent view.
- A view can manage a list of subviews.
- A view can override the size and position of its subviews as needed.
- A view can convert points in its coordinate system to the coordinate systems of other views or the window.

Drawing and animation

- A view draws content in its rectangular area.
- Some view properties can be animated to new values.

Event handling

- A view can receive touch events.
- A view participates in the responder chain.

Adding and Removing Subviews

- Interface Builder is the most convenient way to build view hierarchies
- It can assemble your views graphically, create the relationships between the views, and see exactly how those views will appear at runtime.

Window

Tab bar view

Navigation view



Custom view hierarchy



Code Snippets for CustomViews

- `CGRect viewRect = CGRectMake(0, 0, 100, 100);`
`UIView* myView = [[UIView alloc]`
`initWithFrame:viewRect];`